

Algorithmen 2

Übungen

HINWEIS ZU DEN UNIT TESTS

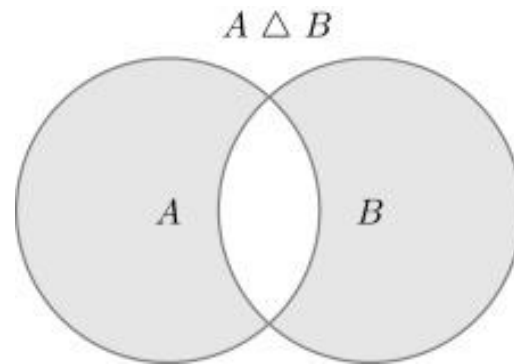
Für alle deine Unit Tests gilt:

- 100 % Testabdeckung
- AAA-Pattern
- Mindestens einen zusätzlichen Edge Case
(d. h. du hast dann theoretisch mehr als 100 % Testabdeckung)

Symmetrische Differenz

Schreibe eine Klasse `ListUtil` mit einer Methode *`symmetricDifference`*, die zwei `Collection<Integer>` erhält.

Die Methode gibt eine neue, **mutable List** zurück, die **keine Duplikate** enthält und nur die Zahlen, die **nicht in beiden Listen** vorkommen (symmetrische Differenz).



Verwende folgenden Unit Test, um deine Implementation zu testen.

Symmetrische Differenz: Unit Test

```
// import static com.google.common.truth.Truth.assertThat;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ListUtilTest {

    @Test
    void testSymmetricDifferenceWithNoCommonElements() {
        List<Integer> list1 = List.of(1, -2, 3);
        List<Integer> list2 = List.of(4, -5, 6);

        List<Integer> actual = ListUtil.symmetricDifference(list1, list2);

        // Hinweis:
        // Da die Elemente wahrscheinlich ursprünglich aus einem Set stammen,
        // ist die Reihenfolge der Elemente zwischen actual und expected nicht
        // garantiert.
        // Aus dem Grund wurde eine Hilfsmethode geschrieben, die zwei Listen
        // auf Gleichheit prüft, aber die Reihenfolge der Elemente ignoriert.
        assertEqualsElementsInAnyOrder(actual, List.of(1, -2, 3, 4, -5, 6));
        // assertThat(actual).containsExactlyElementsIn(List.of(1, -2, 3, 4, -5, 6));
    }

    @Test
    void testSymmetricDifferenceWithSomeCommonElements() {
        List<Integer> list1 = List.of(0, 1, 2, 3, -4);
        List<Integer> list2 = List.of(0, 3, -4, 5);

        List<Integer> actual = ListUtil.symmetricDifference(list1, list2);

        assertEqualsElementsInAnyOrder(actual, List.of(1, 2, 5));
        // assertThat(actual).containsExactlyElementsIn(List.of(1, 2, 5));
    }
}
```

```
@Test
void testSymmetricDifferenceWithEmptyFirstList() {
    List<Integer> empty = List.of();
    List<Integer> expected = List.of(3, 4, 5);

    List<Integer> actual = ListUtil.symmetricDifference(empty, expected);

    assertEqualsElementsInAnyOrder(actual, expected);
    // assertThat(actual).containsExactlyElementsIn(expected);
}

@Test
void testSymmetricDifferenceWithEmptySecondList() {
    List<Integer> expected = List.of(1, 2, 3);
    List<Integer> empty = List.of();

    List<Integer> actual = ListUtil.symmetricDifference(expected, empty);

    assertEqualsElementsInAnyOrder(actual, expected);
    // assertThat(actual).containsExactlyElementsIn(expected);
}

@Test
void testSymmetricDifferenceWithBothEmptyLists() {
    List<Integer> empty1 = List.of();
    List<Integer> empty2 = List.of();

    List<Integer> actual = ListUtil.symmetricDifference(empty1, empty2);

    assertTrue(actual.isEmpty());
    // assertThat(actual).isEmpty();
}
```

```
@Test
void testSymmetricDifferenceWithDuplicates(){
    List<Integer> list1 = List.of(0, 1, 1, 2, 3, -4);
    List<Integer> list2 = List.of(0, 3, 3, -4, 5, 5);

    List<Integer> actual = ListUtil.symmetricDifference(list1, list2);

    assertEqualsElementsInAnyOrder(actual, List.of(1, 2, 5));
    // assertThat(actual).containsExactlyElementsIn(List.of(1, 2, 5));
}

@Test
void testSymmetricDifferenceReturnListIsMutable() {
    List<Integer> list1 = List.of(1);
    List<Integer> list2 = List.of(-2);

    List<Integer> actual = ListUtil.symmetricDifference(list1, list2);
    actual.add(3);

    assertEqualsElementsInAnyOrder(actual, List.of(1, -2, 3));
    // assertThat(actual).containsExactlyElementsIn(List.of(1, -2, 3));
}

// Custom assert method compares the equality of two lists, ignoring the order of the elements.
private static void assertEqualsElementsInAnyOrder(List<Integer> expected, List<Integer> actual) {
    if (expected.size() != actual.size()) {
        String description = "The number of elements in both lists is
different:%nexpected\t%d%nbut was\t\t%d".formatted(expected.size(), actual.size());
        throw new AssertionError(description);
    }

    for (var element : expected) {
        if (frequencyDiffersBetween(element, expected, actual)) {
            String description = "The elements in both lists do not match:%nexpected\t%s%nbut
was\t\t%s".formatted(expected, actual);
            throw new AssertionError(description);
        }
    }
}

private static boolean frequencyDiffersBetween(Integer element, List<Integer> expected,
List<Integer> actual) {
    return Collections.frequency(expected, element) != Collections.frequency(actual, element);
}
}
```

Symmetrische Differenz: Lösung

```
final class ListUtil {  
  
    public static List<Integer> symmetricDifference(Collection<Integer> collection1, Collection<Integer> collection2) {  
        var result = new HashSet<Integer>();  
  
        var leftSide = collection1.stream().filter(number -> !collection2.contains(number)).toList();  
        result.addAll(leftSide);  
  
        var rightSide = collection2.stream().filter(number -> !collection1.contains(number)).toList();  
        result.addAll(rightSide);  
  
        return new ArrayList<>(result);  
    }  
}
```

```
// Alternativ  
public static List<Integer> symmetricDifference(Collection<Integer> collection1, Collection<Integer> collection2) {  
    var leftSet = new HashSet<>(collection1);  
    var rightSet = new HashSet<>(collection2);  
  
    leftSet.removeAll(collection2);  
    rightSet.removeAll(collection1);  
  
    var result = new ArrayList<>(leftSet);  
    result.addAll(rightSet);  
    return result;  
  
    // var result = new ArrayList<Integer>(leftSet.size() + rightSet.size());  
    // result.addAll(leftSet);  
    // result.addAll(rightSet);  
    // return result;  
  
    // return Stream  
    // .concat(leftSet.stream(), rightSet.stream())  
    // .collect(Collectors.toList()); // returns a mutable List.  
  
    // FYI: .toList(); // would return an immutable List!  
}
```

```
// Alternativ  
public static List<Integer> symmetricDifference(Collection<Integer> collection1, Collection<Integer> collection2){  
    Set<Integer> result = new HashSet<>(collection1);  
    Set<Integer> set2 = new HashSet<>(collection2);  
  
    for (var set2Number : set2) {  
        if(result.contains(set2Number)){  
            result.remove(set2Number);  
        } else {  
            result.add(set2Number);  
        }  
    }  
    return new ArrayList<>(result);  
}
```

Duplikate

Implementiere eine Methode, die alle Duplikate aus einer Collection von Integeren entfernt und eine neue Liste ohne Duplikate zurückgibt.

Beispiel:

```
public static void main(String[] args) {  
    List<Integer> numbersWithDuplicates = List.of(1, 2, 3, 2, 4, 5, 1, 6, 7, 8, 9, 4, 10);  
  
    List<Integer> numbersWithoutDuplicates = removeDuplicates(numbersWithDuplicates);  
  
    System.out.println(numbersWithoutDuplicates);  
}
```

Ausgabe:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Duplikate – Lösung

```
public static List<Integer> removeDuplicates(Collection<Integer> inputList) {  
    Set<Integer> uniqueSet = new HashSet<>(inputList);  
    return new ArrayList<>(uniqueSet);  
}
```

Queue<T>

Schreibe eine generische Klasse Queue<T>, die eine einfache Warteschlange (Queue) implementiert. Nutze für die Implementierung eine List.

- Die Warteschlange soll die Methoden add(T e), remove() und isEmpty() haben.
- Neue Elemente werden immer ans Ende hinzugefügt.
- Es wird immer das erste Element entfernt.
- Wenn die Liste leer ist und es wird ein weiteres Element entfernt, soll eine IllegalStateException geworfen werden.

Beispiel:

```
public static void main(String[] args) {  
    Queue<String> queue = new Queue<>();  
  
    queue.add("E1");  
    queue.add("E2");  
    queue.add("E3");  
  
    while (!queue.isEmpty()) {  
        System.out.println(queue.remove());  
    }  
  
    queue.remove(); // Exception  
}
```

Ausgabe:

```
E1  
E2  
E3  
Exception in thread "main"  
java.lang.IllegalStateException: Die  
Warteschlange ist leer.  
    at Queue.remove(Queue.java:17)  
    at Queue.main(Queue.java:37)
```


Queue<T> – Lösung

```
public class Queue<T> {  
    private final List<T> queueList;  
  
    public Queue() {  
        this.queueList = new LinkedList<>();  
    }  
  
    public void add(T element) {  
        queueList.add(element);  
    }  
  
    public T remove() {  
        if (isEmpty()) {  
            throw new IllegalStateException("Die Warteschlange ist leer.");  
        }  
        return queueList.remove(0);  
    }  
  
    public boolean isEmpty() {  
        return queueList.isEmpty();  
    }  
}
```

Matrix transponieren

```
public static void printMatrix(int[][] matrix) {  
    for (int[] row : matrix) {  
        for (int element : row) {  
            System.out.print(element + " ");  
        }  
        System.out.println();  
    }  
}
```

Schreibe eine Methode, die eine gegebene Matrix transponiert (d.h., die Zeilen und Spalten vertauscht).

Beispiel:

```
public static void main(String[] args) {  
    int[][] originalMatrix = {  
        { 1, 2, 3 },  
        { 4, 5, 6 },  
        { 7, 8, 9 }  
    };  
  
    System.out.println("Original Matrix:");  
    printMatrix(originalMatrix);  
  
    int[][] transposedMatrix = transposeMatrix(originalMatrix);  
  
    System.out.println("\nTransponierte Matrix:");  
    printMatrix(transposedMatrix);  
}
```

Ausgabe:

Original Matrix:

```
1 2 3  
4 5 6  
7 8 9
```

Transponierte Matrix:

```
1 4 7  
2 5 8  
3 6 9
```

Matrix transponieren – Lösung

```
public static int[][] transposeMatrix(int[][] matrix) {  
    int rows = matrix.length;  
    int cols = matrix[0].length;  
  
    int[][] transposedMatrix = new int[cols][rows];  
  
    for (int i = 0; i < rows; i++) {  
        for (int j = 0; j < cols; j++) {  
            transposedMatrix[j][i] = matrix[i][j];  
        }  
    }  
  
    return transposedMatrix;  
}
```

Gerade Summe (Stream)

Gegeben ist eine Liste von ganzen Zahlen. Schreibe eine Methode mit Java-Streams, die nur die geraden Zahlen filtert und ihre Summe berechnet.

Beispiel:

```
public static void main(String[] args) {  
    var numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);  
  
    int sum = sumOfEvenNumbers(numbers);  
  
    System.out.println("Sum of even numbers: " + sum);  
}
```

Ausgabe:

```
Sum of even numbers: 30
```

Gerade Summe – Lösung

```
public static int sumOfEvenNumbers(List<Integer> numbers) {  
    return numbers.stream()  
        .filter(n -> n % 2 == 0)  
        .mapToInt(i -> i)  
        // .mapToInt(Integer::intValue)  
        .sum();  
}
```

Personen (Stream)

Gegeben ist eine Liste von Personen mit ihren Namen und Alter. Schreibe eine Methode mit Java-Streams, die das durchschnittliche Alter der Personen berechnet.

Gebe außerdem die Namen der Personen sortiert nach Alter (aufsteigend) aus.

Beispiel:

```
public static void main(String[] args) {  
    List<Person> persons = List.of(  
        new Person("Alice", 25),  
        new Person("Bob", 30),  
        new Person("Charlie", 28),  
        new Person("David", 22));
```

```
    System.out.println("Average age: " + averageAge(persons));  
    System.out.println(namesSortedByAge(persons));  
}
```

```
record Person(String name, int age) {}
```

Ausgabe:

```
Average age: 26.25  
[David, Alice, Charlie, Bob]
```

Personen – Lösung

```
private static double averageAge(List<Person> persons) {  
    return persons.stream()  
        .mapToDouble(p -> p.age())  
        // .mapToDouble(Person::age)  
        .average()  
        .orElse(0.0);  
}
```

```
private static List<String> namesSortedByAge(List<Person> persons) {  
    return persons.stream()  
        .sorted((a, b) -> Integer.compare(a.age(), b.age()))  
        // .sorted(Comparator.comparingInt(Person::age))  
        .map(p -> p.name())  
        .toList();  
}
```